Name: Hamza Ali Ahmed
UEN: 2023UG000142
Course: Advanced Topics in Deep Learning

Assignment I: Knowledge Distillation with Ternary-Weight Quantization-Aware Training

---

Abstract

In this project, three deep learning models were trained and compared for image classification using the CIFAR-10 dataset. The models used were ResNet34, ResNet18 and a ternary version of ResNet18. The ResNet34 model was used as the teacher model, while the normal ResNet18 was used as the baseline. The ternary ResNet18 student was trained using Knowledge Distillation (KD) and Quantization-Aware Training (QAT).

The main aim was to understand how the models perform when their architecture and weight representation are changed. The ResNet34 teacher achieved 94.91% test accuracy, and the ResNet18 baseline achieved 94.58% test accuracy. The ternary student reached 91.03% training accuracy and 88.14% validation accuracy at its best recorded checkpoint in epoch 139.

The results show that the two full-precision models performed better than the ternary student in this experiment. The student also took longer to reach its best recorded validation result. Both ResNet18 models had approximately 11.17 million parameters, compared with 21.28 million in ResNet34.

Overall, this project helped compare the accuracy, training behaviour and parameter count of full-precision and ternary neural networks. It also showed that further work is needed to measure the actual storage savings of the ternary model.

1. Introduction

Deep learning is widely used for tasks such as image classification and object recognition. However, deep learning models can require a large amount of memory and computational power. This can make them difficult to use on devices with limited resources, such as mobile phones and embedded systems.

One way to reduce the computational requirements of a neural network is to use quantization. In a normal full-precision model, weights can have many different floating-point values. In a ternary neural network, the weights used during the forward pass are restricted to three possible values.

In this project, the ternary weights are represented as:

$$Wq \in \{-\alpha, 0, +\alpha\}$$

Here, (W_q) represents the quantized weight, and (\alpha) is a scaling factor that is learned separately for each ternary layer.

Using only three possible weight values can make the model suitable for reduced-precision computation. However, the model may lose some classification accuracy because it has fewer possible weight values available.

To study this, three models were used in the experiment:

1. ResNet34 Teacher: A full-precision model trained from scratch.
2. ResNet18 FP32 Baseline: A smaller full-precision model trained without knowledge distillation.
3. Ternary ResNet18 Student: A model using ternary convolutional and fully connected weights, trained with Knowledge Distillation and Quantization-Aware Training.

The main objective was to compare these models based on their classification accuracy, training behaviour and number of parameters. The experiment also aimed to understand the trade-off between model accuracy and the use of ternary weights.

Model Architectures

ResNet34 Teacher Model

ResNet34 was used as the teacher model. It is a convolutional neural network that uses residual connections to help information and gradients move through the network during training.

For this project, ResNet34 was adjusted for CIFAR-10 images. Instead of using the large initial convolution and max-pooling setup commonly used for ImageNet, the model starts with a (3*3) convolution with a stride of 1. The initial max-pooling layer is also removed to preserve more information from the small (32*32) images.The teacher model has approximately 21.28 million trainable parameters. This model was used as the teacher for knowledge distillation. Its predictions provide additional information that the student model can learn from during training.

ResNet18 Full-Precision Baseline

ResNet18 was used as the second model. It follows the same general residual network design as ResNet34 but contains fewer residual blocks.

The four main stages contain 2, 2, 2 and 2 blocks, respectively. Like the teacher, it uses a (3*3) initial convolution, global average pooling and a final fully connected layer for ten classes.

The model contains approximately 11.17 million trainable parameters.

The ResNet18 baseline was trained from scratch using the original CIFAR-10 labels. Knowledge distillation and ternary quantization were not used for this model.

This baseline helps us understand how a smaller full-precision model performs before introducing ternary quantization and knowledge distillation.

Ternary ResNet18 Student Model

The third model was a ternary version of ResNet18. It uses the same general architecture as the full-precision ResNet18 baseline, but its convolutional and fully connected layers use custom ternary quantization.

In a normal neural network, weights can take different floating-point values. In this model, the weights used during the forward pass are converted into one of three values: negative alpha, zero or positive alpha.

The quantization process first divides each weight by its layer's scaling factor:

$$Wn = W / \alpha$$

The quantization rule is:

$$Q(W) = +\alpha \ \ if\ Wn > 0.5; \quad 0 \ \ if\ -0.5 \leq Wn \leq 0.5; \quad -\alpha \ \ if\ Wn < -0.5$$

Here, (W) is the original weight, (Wn) is the normalized weight, and alpha is a learnable scaling factor.

The scaling factor is learned separately for each ternary layer. This allows different layers to use different weight scales.

The model uses a Straight-Through Estimator (STE) during backpropagation. Since the quantization operation is not normally differentiable, STE provides an approximate gradient that allows the original floating-point weights to continue learning.

Batch normalization layers remain in full precision. The student is trained using both the original CIFAR-10 labels and information from the teacher model.

4. Training Methodology

4.1 Teacher Model Training

The ResNet34 teacher was trained from scratch for 100 epochs. Stochastic Gradient Descent (SGD) was used as the optimizer.

The training configuration included momentum, Nesterov acceleration, weight decay, label smoothing and a cosine learning-rate schedule.

During training, the model's validation performance was checked after each epoch. The checkpoint with the best validation performance was saved and used for the final evaluation.

The best teacher checkpoint was obtained at epoch 96. It achieved 94.91% test accuracy.

4.2 Baseline Model Training

The full-precision ResNet18 baseline was trained separately from scratch. It did not use the teacher's predictions during training.

The optimizer and general training approach were similar to those used for ResNet34. SGD with momentum, Nesterov acceleration, weight decay, label smoothing and cosine learning-rate scheduling were used.

The best checkpoint was selected based on validation accuracy. The model reached its best recorded checkpoint at epoch 98.

This experiment provided a reference for comparing the normal ResNet18 with the ternary ResNet18 student.

4.3 Knowledge Distillation

Knowledge Distillation is a technique in which a smaller student model learns from a larger teacher model.

The teacher provides its prediction scores for all classes, while the student learns from both these predictions and the actual labels in the dataset.

The student uses two loss functions during training: cross-entropy loss and knowledge-distillation loss.

Cross-Entropy Loss

Cross-entropy loss measures the difference between the actual class label and the student's predictions.

It can be written as:

$$LCE = CE(y, ps)$$

Here, (y) is the actual class label and (p_s) is the student's predicted probability distribution.

Knowledge-Distillation Loss

The teacher and student produce prediction scores called logits. These logits are divided by a temperature value (T) before applying the softmax function.

$$psT = softmax(zs / T), \quad ptT = softmax(zt / T)$$

Here, (Zs) represents the student logits, (Zt) represents the teacher logits, and (T) is the temperature.

A higher temperature produces a softer probability distribution. This allows the student to learn more about the teacher's relative confidence across different classes, instead of learning only which class has the highest score.

The knowledge-distillation loss is calculated using the Kullback-Leibler divergence between the teacher and student probability distributions. In this experiment, the loss is multiplied by (T^2).

The final training loss is a weighted combination of the two losses:

$$L = \lambda CE \cdot LCE + \lambda KD \cdot LKD$$

The student experiment used a temperature of T = 4.0 and combined classification and distillation losses throughout training, in conjunction with quantization-aware training so that the ternary forward pass was present from the start of optimization rather than applied only after convergence.

Quantization-Aware Training

Quantization-Aware Training was used to train the student with ternary weights during the forward pass.

Instead of training a normal full-precision network first and applying quantization only at the end, the model uses ternary weights throughout training. This allows the model to adjust to the effects of quantization while learning.

The Straight-Through Estimator allows gradients to pass through the quantization operation approximately. The optimizer can then update the original floating-point weights and the learnable scaling factors.

The overall training process is:

1. Load a batch of training images.

2. Pass the images through the teacher model to obtain its predictions.
3. Pass the same images through the ternary student.
4. Calculate the cross-entropy and knowledge-distillation losses.
5. Combine the two losses and calculate gradients.
6. Update the student weights and scaling factors.
7. Repeat the process for the remaining batches and epochs.

The teacher's parameters remain frozen during student training. Only the student model is optimized.

Experimental Results

The results obtained from the three models are presented below.

Table 1: Comparison of the three models

Model	Best epoch	Training accuracy	Evaluation accuracy
ResNet34 Teacher	96	99.98%	94.91%
ResNet18 FP32 Baseline	98	100.00%	94.58%
Ternary ResNet18 Student	139	91.03%	88.14%*

For the student, 88.14% is the best recorded validation accuracy.

Accuracy Comparison

The ResNet34 teacher achieved 94.91% test accuracy. The full-precision ResNet18 baseline achieved 94.58% test accuracy.

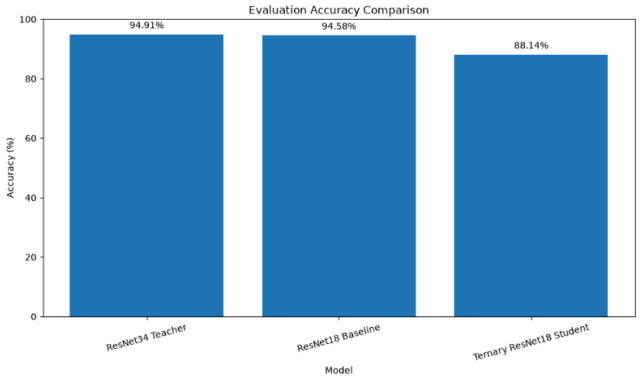
The difference between these two models was only 0.33 percentage points. This shows that the smaller ResNet18 model performed almost as well as the larger ResNet34 model on the CIFAR-10 test set.

The ternary ResNet18 student achieved 88.14% validation accuracy at its best recorded checkpoint. This was lower than the evaluation accuracy of both full-precision models.

The difference between the student's validation accuracy and the baseline's test accuracy was approximately 6.44 percentage points. However, these figures come from different evaluation sets, so the comparison is not completely identical.

The student uses ternary weights during the forward pass, which makes its training process more challenging. Knowledge distillation helps the student learn from the teacher, but it did not completely remove the accuracy difference in this experiment.

Figure 1: Evaluation accuracy comparison of ResNet34 Teacher, ResNet18 FP32 Baseline and Ternary ResNet18 Student.



Training Accuracy and Generalization

Training accuracy shows how well a model classifies the training images. Evaluation accuracy shows how well it performs on a separate dataset.

The ResNet34 teacher achieved 99.98% training accuracy, while the ResNet18 baseline achieved 100.00%. Both models were able to classify almost all the training examples correctly.

The ternary student reached 91.03% training accuracy and 88.14% validation accuracy at its best recorded checkpoint.

The difference between training and evaluation accuracy is called the generalization gap. It can be calculated using the following formula:

$$\textit{Accuracy Gap} = \textit{Training Accuracy} - \textit{Evaluation Accuracy}$$

The student's smaller gap does not automatically mean that it generalizes better. Its training accuracy was also lower than that of the other models. Ternary quantization limits the weight values and can make it harder for the student to fit the training data.

There is also an important difference in the evaluation procedure: the teacher and baseline gaps use test accuracy, whereas the student's gap uses validation accuracy. Therefore, the gap values should be interpreted with this limitation in mind.

Figure 2: Training and evaluation accuracy comparison.

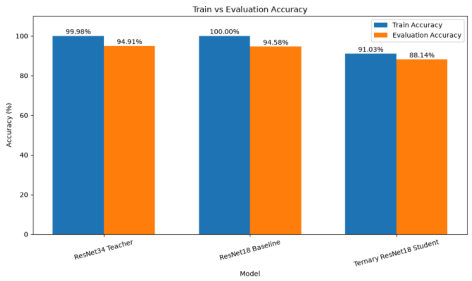
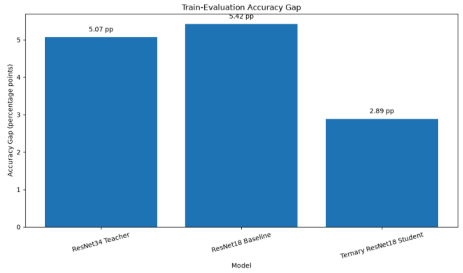


Figure 3: Comparison of the training and evaluation accuracy gaps.



Convergence Behaviour

Convergence refers to the training process reaching a point where the model achieves its best recorded performance.

The ResNet34 teacher reached its best recorded checkpoint at epoch 96. The ResNet18 baseline reached its best checkpoint at epoch 98.

The ternary student reached its best recorded checkpoint at epoch 139. This means it required more training epochs than either full-precision model to reach its best recorded validation performance.

One possible reason is that the student has to learn while its weights are restricted to ternary values during the forward pass. The Straight-Through Estimator also uses an approximate gradient during backpropagation.

These factors make optimization more challenging than training a standard full-precision model.

The results show that the ternary student needed longer training and still achieved a lower recorded evaluation accuracy than the two full-precision models.

Figure 4: Best recorded epoch for each model.

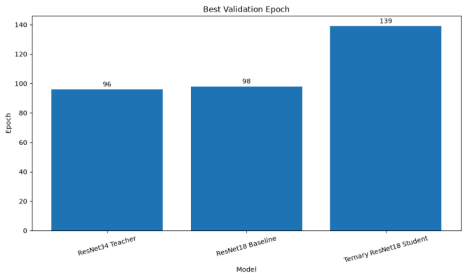
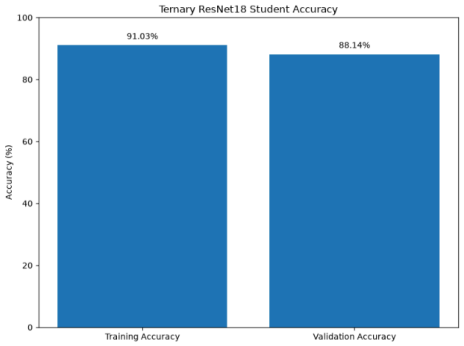


Figure 5: Training and validation accuracy of the ternary ResNet18 student.



6. Model Size and Parameter Comparison

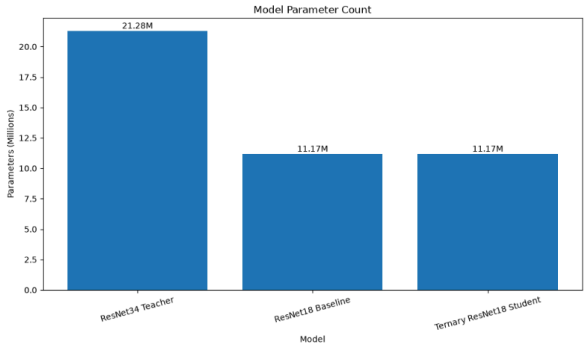
The number of trainable parameters is another important factor when comparing neural networks.

The ResNet34 teacher contains approximately 21.28 million trainable parameters. The ResNet18 baseline and ternary student each contain approximately 11.17 million parameters.

This means that both ResNet18 models have roughly half the number of parameters of the ResNet34 teacher.

The baseline and student have the same reported parameter count because they use the same underlying ResNet18 architecture. Ternary quantization changes how the weights are represented and used during the forward pass, but it does not remove the weights from the architecture.

Figure 6: Trainable parameter count comparison of the three models.



It is important to understand that the number of parameters is not the same as the actual model storage size.

In the current implementation, the ternary student still stores floating-point latent weights. These weights are needed during training so that the optimizer can update them through the Straight-Through Estimator.

The quantized weights used in the forward pass have only three possible values:

$$Wq \in \{-\alpha, 0, +\alpha\}$$

A separate export step would be needed to store the ternary weights in a packed representation. For example, two bits can represent three ternary states, with an additional scaling factor stored for each layer.

However, the actual storage savings were not measured in this experiment. Therefore, it would not be correct to claim a measured reduction in model size based only on the parameter count.

Comparative Discussion

The three models provide different ways to study the effect of architecture size and ternary quantization on classification performance.

The ResNet34 teacher achieved 94.91% test accuracy with approximately 21.28 million parameters. It was the larger of the three models and was used to provide prediction information during knowledge distillation.

The full-precision ResNet18 baseline achieved 94.58% test accuracy with approximately 11.17 million parameters. Its performance was close to that of the teacher even though it had roughly half as many parameters. This suggests that ResNet18 was already capable of performing well on CIFAR-10 without knowledge distillation.

The ternary ResNet18 student also had approximately 11.17 million reported parameters. However, its weights were restricted to three possible values during the forward pass. It achieved 91.03% training accuracy and 88.14% validation accuracy at epoch 139.

Knowledge distillation allowed the student to learn from the teacher's predictions in addition to the actual labels. Quantization-aware training allowed the model to train while using ternary weights during the forward pass. Even with these methods, the student did not reach the recorded accuracy of the full-precision models.

The student also required more epochs to reach its best recorded validation result. This shows that training a quantized model can be more difficult than training a normal full-precision model.

Another important point is that the ternary student and the full-precision baseline have the same parameter count. The advantage of ternary quantization is related to the restricted representation of weights, not a reduction in the number of model parameters.

Since the student checkpoint still stores floating-point latent weights, actual memory savings have not yet been established. A packed ternary export and a measurement of the resulting file size would be needed to evaluate this properly.

Overall, the results show that the smaller full-precision ResNet18 performed very well on CIFAR-10, while the ternary student demonstrated the use of knowledge distillation and quantization-aware training with a lower recorded validation accuracy.

Limitations

The experiment has a few limitations that should be considered when interpreting the results.

1. **Student test accuracy:** The student achieved 88.14% validation accuracy at its best recorded checkpoint. An independent test-set accuracy was not available in the recorded results.
2. **Different evaluation sets:** The teacher and baseline results are test accuracies, while the student's result is validation accuracy. A final test-set evaluation of the student would allow a more direct comparison.
3. **Actual memory savings:** The parameter count does not directly measure compression. The current student checkpoint still contains floating-point latent weights.
4. **Packed ternary representation:** The experiment did not include a separate export process for storing ternary weights in a packed format. As a result, actual storage reduction and runtime memory savings were not measured.
5. **Per-class performance:** Confusion matrices and per-class accuracy metrics were not included because the required final prediction outputs were not available in the recorded results.

These limitations mean that the experiment provides a comparison of the models' recorded classification performance and parameter counts, but it does not yet establish the actual storage or runtime benefits of the ternary model.

Conclusion

In this project, three neural network models were trained and compared using the CIFAR-10 dataset. The models were ResNet34 Teacher, ResNet18 FP32 Baseline and Ternary ResNet18 Student.

The ResNet34 teacher achieved 94.91% test accuracy, while the full-precision ResNet18 baseline achieved 94.58% test accuracy. The small difference between their results shows that ResNet18 can perform well on CIFAR-10 with fewer parameters.

The ternary ResNet18 student was trained using Knowledge Distillation and Quantization-Aware Training. It achieved 91.03% training accuracy and 88.14% validation accuracy at its best recorded checkpoint in epoch 139. It took longer to reach this checkpoint and did not achieve the same recorded evaluation performance as the full-precision models.

The student and baseline both contained approximately 11.17 million reported parameters, while the ResNet34 teacher contained approximately 21.28 million. The ternary student used weights restricted to three possible values during the forward pass, but the saved checkpoint still contained floating-point latent weights.

This project helped demonstrate how knowledge distillation and quantization-aware training can be combined to train a ternary neural network. It also showed that reducing the number of possible weight values can affect classification accuracy and training behaviour.

For future work, the student should be evaluated on the independent test set. The ternary weights should also be exported into a packed representation so that the actual model storage size can be measured. These additional experiments would provide a clearer comparison of accuracy and compression between the full-precision and ternary models.